

Tenjin SDK

Guide d'intégration iOS (Swift)

Documentation technique • Avril 2026

Ce document décrit étape par étape comment intégrer le SDK Tenjin dans une application iOS native développée en Swift. Il couvre l'installation, la configuration, l'initialisation, le tracking d'événements, et la procédure de test.

PARTIE 1 — Installation du SDK

1 Ajouter le SDK Tenjin via Swift Package Manager (SPM)

Dans Xcode, ouvrez votre projet et naviguez vers :

```
File → Add Package Dependencies...
```

Entrez l'URL du repository Tenjin dans le champ de recherche :

```
https://github.com/tenjin/tenjin-ios-sdk
```

Sélectionnez la dernière version stable disponible, puis cliquez sur Add Package. Assurez-vous que le package est bien ajouté à votre app target.

✓ Vous pouvez aussi utiliser CocoaPods en ajoutant pod 'TenjinSDK' dans votre Podfile, puis en exécutant pod install.

2 Créer le Bridging Header (obligatoire pour Swift)

Le SDK Tenjin est écrit en Objective-C. Pour l'utiliser dans un projet Swift, un Bridging Header est nécessaire.

Créer le fichier header

- Dans Xcode : File → New → File From Template... → Header File
- Nommez-le : MonProjet-Bridging-Header.h
- Sous Targets, sélectionnez votre app target

Contenu du fichier Bridging Header

```
// MonProjet-Bridging-Header.h  
  
#import "TenjinSDK.h"
```

Lier le Bridging Header dans Xcode

- Allez dans les Build Settings de votre app target
- Recherchez : Swift Compiler – General
- Dans Objective-C Bridging Header, glissez-déposez votre fichier MonProjet-Bridging-Header.h

3 Configurer Info.plist

Deux clés sont à ajouter dans votre fichier Info.plist.

a) Permission ATT (App Tracking Transparency)

Requis pour iOS 14+ afin d'afficher la popup de consentement au tracking.

```
<key>NSUserTrackingUsageDescription</key>  
<string>Nous utilisons vos données pour vous proposer  
des publicités pertinentes et améliorer votre expérience.</string>
```

b) Endpoint SKAdNetwork (iOS 15+)

Permet à Tenjin de recevoir les postbacks SKAdNetwork directement.

```
<key>NSAdvertisingAttributionReportEndpoint</key>  
<string>https://tenjin-skan.com</string>
```

 **Note** : Ces deux entrées doivent être dans Info.plist au niveau racine, pas dans un sous-dictionnaire.

PARTIE 2 — Création du TenjinService

Créez un nouveau fichier Swift dans votre projet : TenjinService.swift. Ce fichier centralise toutes les interactions avec le SDK.

4 Créer TenjinService.swift

Copiez l'intégralité du code suivant dans votre fichier TenjinService.swift :

```

import Foundation
import AppTrackingTransparency

// MARK: - Configuration

private enum TenjinConfig {
    static let sdkKey = "VOTRE_SDK_KEY_IOS" // Remplacez par votre clé
}

// MARK: - TenjinService

/// Service centralisant toutes les interactions avec le SDK Tenjin.
/// Basé sur : https://tenjin.com/docs/mmp-sdk-ios/
final class TenjinService {

    static let shared = TenjinService()
    private var initialized = false

    private init() {}

    // MARK: - Initialize

    /// Initialise le SDK Tenjin.
    /// Doit être appelé à chaque lancement (didFinishLaunchingWithOptions).
    /// Le userId doit être passé ici pour être envoyé AVANT connect().
    func initialize(userId: String? = nil) async {
        guard !initialized else { return }
        print("TENJIN: Initializing SDK...")

        // 1. Initialiser le SDK
        TenjinSDK.getInstance(TenjinConfig.sdkKey)

        // 2. Définir l'ID utilisateur AVANT connect()
        if let userId = userId {
            TenjinSDK.setCustomerUserId(userId)
            print("TENJIN: Customer User ID set: \(userId)")
        }

        // 3. Demander ATT puis connecter dans le callback
        if #available(iOS 14.0, *) {
            let status = await withCheckedContinuation { continuation in
                ATTrackingManager.requestTrackingAuthorization { status in
                    continuation.resume(returning: status)
                }
            }
            print("TENJIN: ATT status: \(status.rawValue)")
        }

        TenjinSDK.connect()
        initialized = true
        print("TENJIN: SDK Initialized and connected.")
    }

    // MARK: - Customer User ID

    /// Définit l'ID utilisateur pour l'attribution cross-platform.
    /// Préférer le passer directement à initialize() au lancement.
    func setCustomerUserId(_ userId: String) {
        TenjinSDK.setCustomerUserId(userId)
        print("TENJIN: Customer User ID set: \(userId)")
    }
}

```

```

// MARK: - Track Events

/// Envoie un event custom à Tenjin.
/// - eventName : max 80 caractères, max 500 noms uniques.
/// - value      : doit être un entier, sinon Tenjin ignore l'event.
func trackEvent(_ eventName: String, value: Int? = nil) {
    if let value = value {
        TenjinSDK.sendEvent(withName: eventName, andEventValue:
String(value))
    } else {
        TenjinSDK.sendEvent(withName: eventName)
    }
    let valueLog = value.map { " with value: \"($0)\" } ?? ""
    print("TENJIN: Event tracked: \"(eventName)\" \"(valueLog)\"")
}

// MARK: - Track Subscription

/// Track le début d'un free trial / abonnement.
func trackSubscription() {
    TenjinSDK.sendEvent(withName: "start_free_trial")
    if #available(iOS 14.0, *) {
        TenjinSDK.updatePostbackConversionValue(20) // 20 = subscription
    }
    print("TENJIN: Free trial started event tracked")
}

// MARK: - SKAdNetwork Conversion Value

/// Met à jour la conversion value SKAdNetwork.
/// - conversionValue : entier entre 0 et 63.
/// - coarseValue      : "low", "medium" ou "high" (iOS 16.1+ / SKAN 4.0).
/// - lockWindow       : verrouille la fenêtre de conversion (iOS 16.1+).
func updateConversionValue(
    _ conversionValue: Int,
    coarseValue: String? = nil,
    lockWindow: Bool? = nil
) {
    if #available(iOS 16.1, *), let coarseValue = coarseValue {
        if let lockWindow = lockWindow {
            TenjinSDK.updatePostbackConversionValue(
                conversionValue,
                coarseValue: coarseValue,
                lockWindow: lockWindow
            )
        } else {
            TenjinSDK.updatePostbackConversionValue(
                conversionValue,
                coarseValue: coarseValue
            )
        }
    } else if #available(iOS 14.0, *) {
        TenjinSDK.updatePostbackConversionValue(conversionValue)
    }
    print("TENJIN: Conversion value updated: \"(conversionValue)\"")
}
}

```

 **Note :** Remplacez VOTRE_SDK_KEY_IOS par la clé disponible dans le dashboard Tenjin, section Configure → Apps → votre app.

PARTIE 3 — Initialisation au lancement

Le SDK doit être initialisé à chaque lancement de l'application, pas seulement à la première ouverture.

5 Cas A — App SwiftUI (recommandé)

Modifiez votre fichier @main (ex : MonProjetApp.swift) :

```
import SwiftUI

@main
struct MonProjetApp: App {

    var body: some Scene {
        WindowGroup {
            ContentView()
                .task {
                    // Récupérez l'UID de votre système d'auth
                    let uid = await getUserId()
                    // Passer l'uid ici garantit qu'il est envoyé avant connect()
                    await TenjinService.shared.initialize(userId: uid)
                }
        }
    }
}
```

6 Cas B — App UIKit avec AppDelegate

Modifiez votre AppDelegate.swift :

```
import UIKit

@main
class AppDelegate: UIResponder, UIApplicationDelegate {

    func application(
        _ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions:
        [UIApplication.LaunchOptionsKey: Any]?
    ) -> Bool {

        Task {
            let uid = await getUserId()
            await TenjinService.shared.initialize(userId: uid)
        }
    }
}
```

```
    }

    return true
}
}
```

 **Note :** Le SDK doit être initialisé dans `didFinishLaunchingWithOptions` à chaque lancement, y compris les lancements depuis le background.

PARTIE 4 — Tracking des events

Une fois le SDK initialisé, vous pouvez tracker des events depuis n'importe quelle Vue ou ViewController de l'application. Le `TenjinService` est accessible via son singleton partagé.

7 Tracker les events dans vos vues

Event simple (sans valeur)

```
// Depuis n'importe quelle Vue SwiftUI ou ViewController

TenjinService.shared.trackEvent("onboarding_finished")
```

Event avec valeur entière

```
// La valeur DOIT être un entier (Int)
// Si ce n'est pas un entier, Tenjin ignorera l'event
TenjinService.shared.trackEvent("coins_earned", value: 150)
TenjinService.shared.trackEvent("level_reached", value: 5)
```

Abonnement / Free Trial

```
// À appeler après validation de l'achat ou du début du trial
TenjinService.shared.trackSubscription()
```

Exemple complet dans une vue SwiftUI

```
import SwiftUI

struct PaywallView: View {

    var body: some View {
        VStack {
            Button("Commencer l'essai gratuit") {
                // 1. Votre logique d'achat StoreKit...
                // 2. Après validation :
                TenjinService.shared.trackSubscription()
            }
        }
    }
}
```

```

        Button("Voir les niveaux") {
            TenjinService.shared.trackEvent("paywall_dismissed")
        }
    }
    .onAppear {
        TenjinService.shared.trackEvent("paywall_viewed")
    }
}
}

```

Exemple dans un ViewController UIKit

```

class PaywallViewController: UIViewController {

    override func viewDidLoad(_ animated: Bool) {
        super.viewDidLoad(animated)
        TenjinService.shared.trackEvent("paywall_viewed")
    }

    @IBAction func startTrialTapped(_ sender: UIButton) {
        // Votre logique d'achat...
        TenjinService.shared.trackSubscription()
    }
}

```

 **Note :** Les noms d'events sont limités à 80 caractères maximum. Ne dépassez pas 500 noms d'events uniques dans l'ensemble de votre application.

Règles importantes sur les events

- N'envoyez jamais d'events **avant** l'appel à `initialize()` / `connect()`.
- La valeur (value) doit impérativement être un entier. Une valeur décimale sera ignorée.
- Le même eventName peut être envoyé plusieurs fois — Tenjin l'agrège comme compteur ou somme.

PARTIE 5 — Ajouter un Test Device et vérifier les events

Avant de publier, vérifiez que le SDK remonte bien les events dans le dashboard Tenjin en ajoutant votre appareil comme Test Device.

8 Activer les logs de debug dans le code

Ajoutez cet appel juste après `TenjinSDK.getInstance(...)` dans votre `TenjinService`, en environnement de développement uniquement :

```

// Dans initialize(), juste après TenjinSDK.getInstance(TenjinConfig.sdkKey)
#if DEBUG
TenjinSDK.debugLogs()

```

```
#endif
```

Les logs apparaîtront dans la console Xcode et vous permettront de confirmer que le SDK s'initialise et envoie bien les requêtes.

9

Récupérer l'IDFA / Advertising ID de votre appareil

Pour ajouter votre appareil comme Test Device, Tenjin a besoin de votre Advertising ID (IDFA).

- Téléchargez l'application "My Device ID by AppsFlyer" sur l'App Store
- Ouvrez-la sur votre appareil physique
- Copiez la valeur affichée (format UUID)

 **Note :** L'IDFA ne fonctionne que sur un appareil physique. Le simulateur retourne toujours des zéros et ne peut pas être utilisé comme Test Device.

10

Ajouter le Test Device dans le dashboard Tenjin

- Connectez-vous sur dashboard.tenjin.com
- Dans le menu de gauche : Support → Test Devices
- Cliquez sur Add Test Device
- Collez votre Advertising ID dans le champ IDFA
- Donnez un nom à l'appareil (ex : iPhone 15 Pro — Dev)
- Cliquez sur Save

 Une fois ajouté, votre appareil est reconnu par Tenjin comme device de test. Les events envoyés depuis cet appareil apparaîtront dans le Live Event Tool en temps réel.

11

Vérifier les events en temps réel

- Dans le dashboard Tenjin : Support → Live Event Tool
- Sélectionnez votre application dans le menu déroulant
- Lancez votre app sur l'appareil physique via TestFlight
- Déclenchez les events que vous souhaitez tester (ex : ouvrez le Paywall, appuyez sur le bouton d'abonnement)
- Les events doivent apparaître dans le Live Event Tool en quelques secondes

Avant de soumettre votre app sur l'App Store, assurez-vous que les logs debug sont conditionnels à la configuration DEBUG :

```
//  Correct : logs activés uniquement en debug
#if DEBUG
TenjinSDK.debugLogs()
#endif

//  Incorrect : ne jamais laisser ça en production
// TenjinSDK.debugLogs()
```
